

LeetCode 826: Most Profit Assigning Work

Target: Netease, DoorDash

Question 199 in Study Notes

Problem Overview & Arrays Provided

The problem introduces **3 core arrays**:

Array Name	Description
profit	The profit generated by completing a specific job.
difficulty	The difficulty level associated with each job.
worker	The skill levels of individual workers. Each worker element contains only those elements indicating what maximum difficulty that specific worker can handle.

Core Rules & Conditions

- **Condition I:** A worker can do a job only if their skill $\geq$ difficulty of the job
- **Condition II:** Each worker should perform the **most profitable job** whose difficulty $\leq$ worker's skill. That is, a job of **AT MOST** x difficulty.

1. Brute Force Approach — $O(N \times M)$

Iterate through every worker's skill level and compare it against all available jobs to locate the maximum profit option.

Pseudocode:

```
for (int skill : worker) {  
    int best = 0;  
    for (int i = 0; i < n; i++) {  
        if (skill >= difficulty[i]) {  
            best = max(best, profit[i]);  
        }  
    }  
    ans += best;  
}  
return ans;
```

2. Optimal Approach — Two Pointers & Sorting

As a worker's skills increase, more profitable job opportunities systematically open up.

- **Step 1:** Pair up and **sort jobs by difficulty**.
- **Step 2:** **Sort workers by their skill level**.
- **Step 3:** Use a persistent pointer tracking the maximum available profit up to the current skill level. Since workers are sorted, the next worker retains all opportunities available to the previous one, plus potential new additions. No loop resets are needed!

Optimal Implementation (C++)

```
class Solution {
public:
    int maxProfitAssignment(vector<int>& difficulty, vector<int>& profit, vector<int>& worker) {
        int n=difficulty.size();
        int m=worker.size();
        vector<pair<int,int>> job;
        for(int i=0;i<n;i++){
            job.push_back({difficulty[i],profit[i]});
        }

        sort(job.begin(),job.end());
        sort(worker.begin(),worker.end()); //taaki jaise jaise skill badhe job open krsktte hain

        int ans=0;
        int l=0;
        int maxu=0; //har loop me reset krne ki zrurat nhi cause ek worker jo pichla job krchka h
        for(int labour:worker){
            while(l<n && job[l].first<=labour){
                maxu=max(maxu,job[l].second);
                l++;
            }
            ans+=maxu;
        }

        return ans;
    }
};
```

Authored by: Arpit Raj | Institution: LNMIIT Jaipur | Date: July 7, 2026

All rights reserved. Unauthorized reproduction prohibited.